

Spring Framework is an **open-source Java framework** used to build **enterprise-level applications**.

The core idea behind Spring is:

- **IoC (Inversion of Control)** → Spring controls object creation
- **DI (Dependency Injection)** → Dependencies are injected, not created manually

Instead of writing tightly coupled code like:

```
Car car = new Car();
```

Spring does:

```
@Autowired
```

```
Car car;
```

So, objects are **managed by Spring container** (ApplicationContext).

From your notes:

- Spring acts as a **container** that manages objects (beans) and their lifecycle
- It reduces **manual coding, tight coupling, and complexity**

Why Spring is Used

Spring is used to:

1. **Reduce Tight Coupling**
 - Classes depend on interfaces, not actual objects
2. **Manage Objects Automatically**
 - Spring container creates and manages beans
3. **Simplify Enterprise Development**
 - Provides ready features (AOP, JDBC, MVC, Security)
4. **Improve Code Maintainability**
 - Clean layered architecture (Controller → Service → DAO)
5. **Support Multiple Modules**
 - Spring MVC, Spring Boot, Spring Security, Spring Data JPA

Interview Answer (Perfect Answer)

“Spring Framework is an open-source Java framework used to build enterprise applications. It follows IoC and Dependency Injection principles to reduce tight coupling and manage objects using a container. It simplifies development by providing features like AOP, MVC, and database integration

IoC (Inversion of Control) is a **design principle** where the control of **object creation and dependency management is given to the Spring container**, instead of the developer.

👉 Normally (without IoC):

```
class Service {  
    Repository repo = new Repository(); // object created manually  
}
```

- Class itself creates dependency → **tight coupling**

👉 With IoC (Spring):

```
class Service {  
    @Autowired  
    Repository repo; // Spring provides object  
}
```

- Spring container creates and injects → **loose coupling**

So, control is **inverted (shifted)**:

- From **developer** → **Spring container**
-

Key Idea

- You **don't create objects**
 - You **just use them**
 - Spring handles:
 - Object creation
 - Dependency injection
 - Lifecycle
-

Interview Answer (Exact)

“Inversion of Control is a design principle where the control of object creation and dependency management is transferred from the developer to the Spring container, which helps achieve loose coupling and better maintainability.”

Dependency Injection (DI) is a technique where **required dependencies (objects)** are provided to a **class by the Spring container**, instead of the class creating them itself.

👉 Without DI:

```
class Service {  
    Repository repo = new Repository(); // manually created  
}
```

- Tight coupling (hard to change/test)

👉 With DI:

```
class Service {  
    @Autowired  
    Repository repo; // injected by Spring  
}
```

- Loose coupling (flexible & testable)
-

Key Benefit

- Promotes **loose coupling**
 - Improves **testability**
 - Makes code **flexible and maintainable**
-

Interview Answer (Exact)

“Dependency Injection is a technique where the Spring container provides required dependencies to a class instead of the class creating them, helping to achieve loose coupling and better maintainability.

Dependency Injection is performed by the Spring Container, not the user.

- You (developer) only **declare the dependency**
- Spring container **creates the object and injects it**

👉 Example:

```
class Service {  
    @Autowired  
    Repository repo; // you declare  
}
```

- You did NOT create new Repository()
- Spring container:
 - Creates Repository object
 - Injects it into Service

So responsibility is:

- **User → defines dependency**
 - **Spring → injects dependency**
-

Interview Answer (Exact)

“Dependency Injection is performed by the Spring container. The developer only declares the dependencies, and the container creates and injects the required objects automatically.”

Spring Context is the **container in Spring that manages all the objects (beans) and their lifecycle.** It is implemented using **ApplicationContext**.

👉 What it does:

- Creates objects (beans)
- Injects dependencies (DI)
- Manages bean lifecycle (init → destroy)
- Provides features like AOP, events, etc.

👉 Example:

```
ApplicationContext context =  
    new AnnotationConfigApplicationContext(AppConfig.class);
```

```
Service service = context.getBean(Service.class);
```

Here:

- context = Spring container
- It creates and manages Service object

“Spring Context is the container in Spring that manages beans and their lifecycle, handles dependency injection, and provides configuration and runtime support for the application

Both **BeanFactory** and **ApplicationContext** are **Spring containers** used to manage beans.

BeanFactory (Basic Container)

- **Lazy loading** → bean is created only when requested
- **Basic features only**
- No support for advanced features like AOP, events
- Used in **simple or memory-sensitive applications**

👉 Example:

```
BeanFactory factory = new XmlBeanFactory(new ClassPathResource("config.xml"));
```

ApplicationContext (Advanced Container)

- **Eager loading** → beans are created at startup
- Provides **advanced features**:
 - AOP support
 - Event handling
 - Internationalization (i18n)
- Most commonly used in real projects

👉 Example:

```
ApplicationContext context =  
    new ClassPathXmlApplicationContext("config.xml");
```

Key Differences

Feature	BeanFactory	ApplicationContext
Bean Loading	Lazy	Eager
Performance	Slightly faster start	Slower start (loads all beans)
Features	Basic	Advanced (AOP, events)
Usage	Rare	Widely used

“BeanFactory is a basic Spring container that uses lazy initialization and provides minimal features, whereas ApplicationContext is an advanced container that uses eager initialization and supports features like AOP, event handling, and internationalization. ApplicationContext is commonly used in real-world applications.”

Question: Explain Spring Bean lifecycle.

Spring Bean lifecycle is the **process from creation to destruction of a bean managed by the Spring container.**

Steps:

1. **Bean Creation**
 - Spring creates the bean using constructor
2. **Dependency Injection**
 - Required dependencies are injected
3. **Initialization**
 - Custom logic runs after injection
 - Using:

@PostConstruct

4. **Bean Ready to Use**
 - Bean is fully initialized and used in application
5. **Destruction**
 - When application shuts down, cleanup happens
 - Using:

@PreDestroy

“Spring Bean lifecycle includes creation of the bean, dependency injection, initialization using methods like @PostConstruct, making the bean ready for use, and destruction using @PreDestroy when the application shuts down

Question: What are the different scopes of Spring Beans?

Spring Bean scope defines **how many instances of a bean are created and how they are managed.**

Types of Bean Scopes:

1. **Singleton (Default)**
 - Only **one instance** per application
 - Shared across entire container
2. **Prototype**
 - **New instance every time** it is requested
3. **Request**
 - One bean per **HTTP request**
4. **Session**
 - One bean per **user session**
5. **Application (Context)**
 - One bean per **ServletContext (web app)**
6. **Global**
 - Single instance shared across **multiple applications/sessions**
 - Mainly used in **portlet-based environments** (rarely used)

“Spring Bean scopes define how beans are created and managed. The main scopes are Singleton, Prototype, Request, Session, Application, and Global, where Singleton creates a single shared instance and Prototype creates a new instance for each request”

Question: How do you define a custom scope for a Spring bean?

A **custom scope** is created when the existing scopes (singleton, prototype, etc.) are not enough for your requirement.

Steps:

1. Create a Custom Scope Class

- Implement Scope interface
- Override methods like:
 - get()
 - remove()
 - registerDestructionCallback()

```
class MyScope implements Scope {  
    public Object get(String name, ObjectFactory<?> objectFactory) {  
        return objectFactory.getObject();  
    }  
    public Object remove(String name) { return null; }  
}
```

2. Register the Custom Scope

```
@Bean  
public static CustomScopeConfigurer customScopeConfigurer() {  
    CustomScopeConfigurer configurer = new CustomScopeConfigurer();  
    configurer.addScope("myscope", new MyScope());  
    return configurer;  
}
```

3. Use the Custom Scope

```
@Component  
@Scope("myscope")  
class MyBean {  
}
```

“A custom scope in Spring is defined by implementing the Scope interface, registering it using CustomScopeConfigurer, and then applying it to a bean using the @Scope annotation”

Question: What is @Component?

@Component is a **stereotype annotation** used to mark a class as a **Spring Bean**.

- Spring automatically **detects it during component scanning**
- Creates and manages it inside the **Spring container**
- Enables **dependency injection**

👉 Example:

```
@Component  
class Service {  
}
```

- Spring will create this class as a bean automatically

“@Component is a stereotype annotation used to mark a class as a Spring-managed bean, allowing Spring to automatically detect and create it during component scanning

Question: What is @Service?

@Service is a **stereotype annotation** used to mark a class as a **service layer bean** in Spring.

- It is a specialization of @Component
- Used for **business logic layer**
- Spring automatically **detects and creates it as a bean**

👉 Example:

```
@Service  
class UserService {  
}
```

“@Service is a Spring stereotype annotation used to mark a class as a service layer component containing business logic, and it is automatically detected and managed as a bean by the Spring container

Question: What is @Repository?

@Repository is a **stereotype annotation** used to mark a class as a **data access layer (DAO) component** in Spring.

- It is a specialization of @Component
- Used for **database operations**
- Enables **exception translation** (converts JDBC exceptions into Spring exceptions)
- Automatically detected and managed as a bean

👉 Example:

```
@Repository  
class UserRepository {  
}
```

“@Repository is a Spring stereotype annotation used to mark a class as a data access component, and it enables automatic exception translation and bean management by the Spring container

@Controller is a **stereotype annotation** used in Spring MVC to handle **web requests**.

- It is a specialization of @Component
- Used in the **presentation layer**
- Handles HTTP requests and returns **view names (HTML/JSP)**
- If you want to return data (JSON), you need @ResponseBody

👉 Example:

@Controller

```
class MyController {
```

```
    @GetMapping("/home")
    public String home() {
        return "home"; // returns view
    }
}
```

“@Controller is a Spring annotation used to handle web requests and return views in a Spring MVC application.”

Question: What is the role of stereotype annotations?

Stereotype annotations (@Component, @Service, @Repository, @Controller) are used to **mark classes as Spring-managed beans and define their role in the application layer**.

- Enable **automatic component scanning**
- Tell Spring to **create and manage objects (beans)**
- Help in organizing code into layers:
 - Controller → presentation
 - Service → business logic
 - Repository → data access

“Stereotype annotations are used to mark classes as Spring-managed beans and define their role in different layers of the application, enabling automatic detection and dependency injection.”

Question: What happens if a class is not annotated?

If a class is **not annotated** (and not defined as a bean manually):

- Spring will **NOT detect it during component scanning**
- No bean will be created in the container
- **Dependency Injection will fail**
- You may get errors like:
 - NoSuchBeanDefinitionException
 - NullPointerException (if used without injection)

“If a class is not annotated or not defined as a bean, Spring will not detect or manage it, and dependency injection will fail, leading to runtime errors.”

Question: What is @Bean?

@Bean is an annotation used to **manually create and register a bean in the Spring container**.

- Used inside a **@Configuration** class
- Gives **full control over object creation**
- Used when:
 - Class is from **third-party library**
 - You need **custom initialization**

👉 Example:

@Configuration

class AppConfig {

 @Bean

 public Service service() {

 return new Service();

 }

}

- Spring will create and manage this object as a **bean**

“@Bean is used to manually define and register a bean in the Spring container inside a configuration class, giving full control over object creation.”

Question: What is @ComponentScan?

@ComponentScan is used to **tell Spring where to scan for classes annotated with stereotype annotations** like @Component, @Service, @Repository, and @Controller.

- Spring scans the specified package
- Automatically detects annotated classes
- Creates and registers them as **beans**

👉 Example:

@Configuration

@ComponentScan("com.app")

class AppConfig {

}

- Spring will scan com.app package and its sub-packages

“@ComponentScan is used to specify the packages that Spring should scan to automatically detect and register beans annotated with stereotype annotations.”

Question: What is @Autowired?

@Autowired is used for **automatic dependency injection** in Spring.

- Tells Spring to **inject the required bean automatically**
- Works by **type (default)**
- Can be used on:
 - **Constructor**
 - **Setter**
 - **Field**

👉 Example:

@Autowired

private Repository repo;

- Spring finds matching bean and injects it

“@Autowired is used for automatic dependency injection in Spring, where the container automatically injects the required bean based on type.”

Question: What is the difference between @Autowired, @Resource, and @Inject?

All three are used for **dependency injection**, but they differ in how they resolve beans.

@Autowired (Spring)

- Injects **by type (default)**
- If multiple beans → use @Qualifier
- Can be used on constructor, setter, field

@Resource (Java - JSR 250)

- Injects **by name (default)**
- Matches variable name with bean name
- If name not found → falls back to type

@Inject (Java - JSR 330)

- Similar to @Autowired
- Injects **by type**
- Uses @Named instead of @Qualifier

Key Difference

- @Autowired → by **type** (Spring specific)
- @Resource → by **name** (Java standard)
- @Inject → by **type** (Java standard)

“@Autowired is a Spring annotation that injects dependencies by type, @Resource is a Java annotation that injects by name, and @Inject is a Java standard annotation similar to @Autowired that injects by type.

Question: What happens when two beans of the same type exist?

When two beans of the **same type** exist and you use @Autowired:

- Spring finds **multiple matching beans**
 - It gets confused and throws:
NoUniqueBeanDefinitionException
-

👉 Example:

@Component

class Repo1 implements Repository {}

@Component

class Repo2 implements Repository {}

@Autowired

Repository repo; // ERROR

When two beans of the same type exist:

- Spring finds **more than one match**
 - It cannot decide which one to inject
 - So it throws **NoUniqueBeanDefinitionException**
-

Why solution works

1. @Qualifier

@Autowired

@Qualifier("repo1")

Repository repo;

- Here you are **explicitly telling Spring**:
👉 “Inject the bean named repo1”
 - So ambiguity is removed
 - Spring stops guessing → directly injects correct bean
-

2. @Primary

@Primary

@Component

class Repo1 implements Repository {}

- Here you are marking **one bean as default**
 - When multiple beans exist:
👉 Spring automatically picks the **@Primary bean**
 - No need to specify every time
-

Key Understanding

- Problem = Spring has **multiple choices**
- Solution:

- @Qualifier → **you choose manually**
- @Primary → **you set default choice**

“When multiple beans of the same type exist, Spring throws NoUniqueBeanDefinitionException because of ambiguity. This is resolved using @Qualifier to specify the exact bean or @Primary to mark a default bean.

Question: What is @Qualifier?

@Qualifier is used to **specify exactly which bean to inject when multiple beans of the same type exist.**

👉 Example:

@Component

class Repo1 implements Repository {}

@Component

class Repo2 implements Repository {}

@Autowired

@Qualifier("repo1")

Repository repo;

- Two beans exist → Repo1, Repo2
- @Qualifier("repo1") tells Spring:

👉 “Inject Repo1 only”

Key Idea

- Used with @Autowired
 - Removes **ambiguity when multiple beans exist**
-

“@Qualifier is used to specify the exact bean to inject when multiple beans of the same type exist.”

Question: What is @Primary?

@Primary is used to **mark one bean as the default choice when multiple beans of the same type exist.**

👉 Example:

@Component

@Primary

class Repo1 implements Repository {}

@Component

class Repo2 implements Repository {}

@Autowired

Repository repo;

- Two beans exist → Repo1, Repo2
- Spring automatically picks **Repo1** because it is marked @Primary

Key Idea

- Used when multiple beans exist
- Sets a **default bean**
- Avoids using @Qualifier everywhere

“@Primary is used to mark a bean as the default choice when multiple beans of the same type exist, so Spring injects it automatically.”

Question: What happens if @Qualifier is not used?

If multiple beans of the **same type** exist and @Qualifier is not used:

- Spring finds **more than one matching bean**
- It cannot decide which one to inject
- Throws **NoUniqueBeanDefinitionException**

👉 Example:

@Component

class Repo1 implements Repository {}

@Component

class Repo2 implements Repository {}

@Autowired

Repository repo; // ERROR

Special Case

- If one bean is marked @Primary
→ Spring will inject that bean (no error)

“If @Qualifier is not used and multiple beans of the same type exist, Spring throws NoUniqueBeanDefinitionException unless one bean is marked as @Primary.”

Question: How does Spring handle circular dependencies?

Circular dependency = two beans depend on each other.

👉 Example:

@Component

class A {

 @Autowired

 B b;

```
}
```

```
@Component
```

```
class B {
```

```
    @Autowired
```

```
    A a;
```

```
}
```

How Spring handles it

- **Setter / Field Injection**

- Spring can resolve it
- Uses **early bean creation (partially initialized objects)**
- Injects reference before full initialization

- **Constructor Injection**

```
class A {
```

```
    public A(B b) {}
```

```
}
```

```
class B {
```

```
    public B(A a) {}
```

```
}
```

- Spring **CANNOT** resolve
- Throws:
BeanCurrentlyInCreationException

How Spring solves:

1. Create object of A (empty, no values yet)
2. Create object of B
3. Inject B into A
4. Inject A into B

✓ Because Spring creates **object first, then injects later**

Case 2: Constructor Injection (Fails)

```
class A {
```

```
    public A(B b) {}
```

```
}
```

```
class B {
```

```
    public B(A a) {}
```

```
}
```

Problem:

- To create A → need B
- To create B → need A
- Both waiting → **deadlock**

✗ Spring throws error

Key Idea

- Setter / Field → **works**
 - Constructor → **fails**
-

“Spring handles circular dependencies using setter or field injection by creating beans in multiple steps, but it cannot resolve circular dependencies with constructor injection and throws `BeanCurrentlyInCreationException`”

Question: What is Spring AOP?

Spring AOP (Aspect-Oriented Programming) is used to **separate cross-cutting concerns** (common logic used in many places) from the main business logic.

Why AOP is needed

Without AOP:

```
public void transfer() {  
    System.out.println("Logging");    // repeated  
    checkSecurity();                  // repeated  
    // business logic  
}
```

👉 Same code (logging, security) gets **repeated in many methods**

👉 Leads to:

- Code duplication
 - Hard maintenance
-

With AOP

You write common logic **once** and apply it everywhere.

@Aspect

@Component

class LoggingAspect {

@Before("execution(* Service.*(..))")

public void log() {

System.out.println("Logging...");

}

}

- Automatically runs before all service methods
 - No need to write logging in every method
-

Core Concepts

1. Aspect

- Class that contains cross-cutting logic
- Example: LoggingAspect

2. Advice

- Action performed
Types:
 - @Before → before method
 - @After → after method
 - @AfterReturning → after success
 - @AfterThrowing → after exception
 - @Around → before + after

3. Pointcut

- Defines **where** to apply logic
- Example:

`execution(* Service.*(..))`

4. Join Point

- Specific point in execution (method call)

5. Target

- Actual business class (Service)

6. Proxy

- Spring creates a proxy object
 - Adds extra behavior (logging, etc.)
-

Flow (Important)

1. Client calls method
 2. Proxy intercepts call
 3. Advice (logging/security) executes
 4. Actual method runs
-

Key Benefits

- Removes duplicate code
 - Improves maintainability
 - Clean separation of concerns
-

“Spring AOP is used to separate cross-cutting concerns like logging, security, and transactions from business logic using aspects, advice, and pointcuts, improving code modularity and maintainability

Question: How is Spring AOP different from OOP?

OOP (Object-Oriented Programming)

- Focuses on **structuring code using classes and objects**
- Uses concepts like:
 - Encapsulation
 - Inheritance
 - Polymorphism
- Handles **business logic inside classes**

👉 Problem:

- Common logic (logging, security) gets **repeated across multiple classes**
-

AOP (Aspect-Oriented Programming)

- Focuses on **separating cross-cutting concerns**
- Moves common logic **outside business classes**
- Applies it **dynamically using aspects**

👉 Solves:

- Code duplication
 - Code scattering
 - Code tangling
-

Key Difference

- OOP → organizes **what the program does (business logic)**
 - AOP → handles **common repeated logic separately**
-

Simple Example

Without AOP (OOP only):

```
public void save() {  
    log();  
    // business logic  
}
```

With AOP:

```
@Before("execution(* Service.*(..))")  
public void log() { }
```

“OOP focuses on structuring code using classes and objects to handle business logic, whereas AOP focuses on separating cross-cutting concerns like logging and security from business logic to reduce code duplication and improve modularity.”

What is logging?

Logging means **recording information about what is happening inside an application** while it is running.

Simple understanding

It is like **tracking or printing messages** to know:

- What method is executed
 - What data is coming
 - Whether error occurred
-

Example

```
System.out.println("User logged in");
```

or using logger:

```
log.info("User logged in");
```

Why logging is used

- To **debug issues**
 - To **track application flow**
 - To **monitor errors and performance**
-

In AOP

Logging is a **cross-cutting concern** because:

- It is needed in many methods
- Instead of writing everywhere → AOP handles it in one place

Think like this:

- **Business logic = Main story (movie)** 🎬
- **Aspect = Background tasks (lighting, sound, subtitles)** 🎧

The movie works without them, but they **improve and manage behavior globally**.

“Logging is the process of recording application events, messages, or errors during execution to help in debugging, monitoring, and tracking the application flow.”

Question: What is a proxy in Spring AOP?

A **proxy** is an **object created by Spring** that sits between the client and the actual target object.

- It **intercepts method calls**
 - Applies AOP logic (logging, security, transactions)
 - Then calls the **actual method**
-

Simple Flow

Client → **Proxy** → Target Object

- Proxy adds extra behavior before/after method execution
-

Example

```
service.save();
```

Internally:

1. Call goes to **proxy**
2. Proxy executes **advice (logging, etc.)**
3. Then calls actual save() method

Key Idea

- Proxy = **wrapper around original object**
- Adds extra functionality **without changing original code**

“A proxy in Spring AOP is an object created by Spring that intercepts method calls, applies cross-cutting concerns like logging or security, and then delegates the call to the actual target object.”

Question: Difference between JDK Dynamic Proxy and CGLIB Proxy

JDK Dynamic Proxy

- Works only with **interfaces**
- Creates proxy using **interface implementation**
- Target class **must implement an interface**

👉 Example:

```
interface Service {  
    void save();  
}
```

CGLIB Proxy

- Works with **classes (no interface needed)**
- Creates proxy by **extending the target class** (inheritance)

👉 Example:

```
class Service {  
    void save() {}  
}
```

Key Differences

- JDK Proxy → **interface-based**
- CGLIB → **class-based (extends class)**
- JDK → cannot work without interface
- CGLIB → works without interface
- JDK → slightly lighter
- CGLIB → slightly heavier (uses bytecode generation)

Important (Interview Point)

- If **interface is present** → Spring uses **JDK Proxy**
- If **no interface** → Spring uses **CGLIB**

“JDK Dynamic Proxy works with interfaces and creates proxies based on interface implementation, whereas CGLIB works with classes and creates proxies by extending the target class. Spring uses JDK proxy if an interface is present, otherwise it uses CGLIB

Question: Explain internal working of Spring AOP

Step-by-step flow

1. Spring detects AOP configuration

- Finds classes annotated with @Aspect
-

2. Creates Proxy Object

- Instead of giving actual object, Spring creates a **proxy**
 - Uses:
 - JDK Proxy (if interface present)
 - CGLIB (if no interface)
-

3. Client calls method

service.save();

- Call goes to **proxy**, not actual object
-

4. Proxy intercepts the call

- Checks if any **advice is applicable** (based on pointcut)
-

5. Advice execution

- Executes AOP logic:
 - @Before
 - @After
 - @Around, etc.
-

6. Actual method execution

- Proxy calls the **target method**
-

7. After execution

- Runs remaining advice (like @After, @AfterReturning)
-

Simple Flow

Client → Proxy → Advice → Target Method → Advice → Result

Key Idea

- Spring does NOT modify original class
- It adds behavior using **proxy at runtime**

“Spring AOP works by creating a proxy object for the target bean. When a method is called, the proxy intercepts the call, applies the defined advice based on pointcuts, and then delegates the call to the actual method, enabling cross-cutting concerns without modifying the original code.”

with example?

Question: Explain internal working of Spring AOP (with example)

Example

@Service

```
class PaymentService {
```

```
    public void pay() {
```

```
        System.out.println("Payment done");
```

```
    }
```

```
}
```

@Aspect

@Component

```
class LoggingAspect {
```

```
    @Before("execution(* PaymentService.pay(..))")
```

```
    public void log() {
```

```
        System.out.println("Logging before payment");
```

```
    }
```

```
}
```

Internal Working (Step-by-step)

1. Spring starts

- Detects @Aspect and @Service
-

2. Proxy is created

- Instead of original PaymentService, Spring creates a **proxy object**
-

3. Client calls method

```
paymentService.pay();
```

4. Call goes to proxy

- Not directly to PaymentService
-

5. Proxy checks pointcut

- Matches:

```
execution(* PaymentService.pay(..))
```

6. Advice executes

Logging before payment

7. Actual method executes

Payment done

Output

Logging before payment

Payment done

Final Flow

Client → Proxy → @Before → Target Method → Result

“In Spring AOP, a proxy is created for the target bean. When a method is called, the proxy intercepts the call, executes advice like logging based on pointcuts, and then calls the actual method, as shown in the example where logging runs before the payment method.”

Question: How does Spring handle transactions?

Spring handles transactions using **AOP (proxy-based mechanism)** and **transaction manager**.

Example

@Service

class PaymentService {

 @Transactional

 public void transfer() {

 debit();

 credit();

 }

}

Internal Working (Step-by-step)

1. Spring creates a proxy

- For PaymentService because of @Transactional
-

2. Client calls method

paymentService.transfer();

3. Call goes to proxy

4. Transaction starts

- Proxy calls **TransactionManager.begin()**
-

5. Business logic executes

```
debit();  
credit();
```

6. If everything is successful

- Proxy calls **commit()**
-

7. If exception occurs

- Proxy calls **rollback()**
-

Simple Flow

Client → Proxy → Start Transaction → Method → Commit / Rollback

Key Idea

- Spring uses **AOP + Proxy**
 - You don't write transaction code manually
 - Just use `@Transactional`
-

“Spring handles transactions using a proxy-based AOP mechanism. When a method annotated with `@Transactional` is called, Spring starts a transaction, executes the method, and then commits or rolls back the transaction based on success or failure.”

Question: What is `@Transactional`?

`@Transactional` is used to **manage database transactions automatically in Spring**.

- It ensures **all operations in a method are treated as a single unit**
 - If everything is successful → **commit**
 - If any error occurs → **rollback**
-

Example

`@Service`

```
class PaymentService {
```

```
    @Transactional
```

```
    public void transfer() {
```

```
        debit(); // deduct money
```

```
        credit(); // add money
```

```
    }
```

```
}
```

What happens internally

- Spring creates a **proxy**
- When method is called:

1. **Transaction starts**
 2. Method executes
 3. If success → **commit**
 4. If exception → **rollback**
-

Key Idea

- Ensures **data consistency**
 - Avoids partial updates
-

“@Transactional is used in Spring to manage transactions automatically by ensuring that a method executes within a transaction, committing on success and rolling back on failure.”

Question: Difference between @Transactional on class vs method

@Transactional at Class Level

@Transactional

@Service

```
class PaymentService {
```

```
    public void transfer() { }
```

```
    public void refund() { }
```

```
}
```

- Applies to **all methods** in the class
 - Every method becomes **transactional by default**
-

@Transactional at Method Level

@Service

```
class PaymentService {
```

```
    @Transactional
```

```
    public void transfer() { }
```

```
    public void checkBalance() { }
```

```
}
```

- Applies **only to specific method**
 - Other methods are **non-transactional**
-

Important Point

@Transactional

```
class Service {
```



```
@Transactional(propagation = Propagation.REQUIRES_NEW)
public void method() {}
}
```

- **Method-level overrides class-level**
- More specific configuration takes priority

Key Difference

- Class level → applies to **all methods**
- Method level → applies to **specific method only**

“@Transactional at class level applies transaction management to all methods in the class, whereas at method level it applies only to that specific method, and method-level configuration overrides class-level settings.”

Question: Difference between programmatic and declarative transaction management

Programmatic Transaction Management

- Transactions are handled **manually in code**
- Developer controls **begin, commit, rollback**

👉 Example:

```
TransactionStatus status = transactionManager.getTransaction(def);
```

```
try {
    // business logic
    transactionManager.commit(status);
} catch (Exception e) {
    transactionManager.rollback(status);
}
```

- More control
- More complex and verbose

Declarative Transaction Management

- Transactions are handled **automatically by Spring**
- Uses annotations like @Transactional

👉 Example:

```
@Transactional
public void transfer() {
    // business logic
}
```

- Less code
- Easy to use
- Most commonly used

Key Difference

- Programmatic → **manual control (code-based)**
- Declarative → **automatic (annotation-based)**

“Programmatic transaction management requires manual handling of transactions using code, whereas declarative transaction management uses annotations like @Transactional to automatically manage transactions.”

Question: What is Spring JDBC?

Spring JDBC is a module of Spring that **simplifies database interaction using JDBC.**

- Removes boilerplate code like:
 - Opening/closing connections
 - Handling exceptions
- Provides easy APIs like **JdbcTemplate**

Example

@Autowired

JdbcTemplate jdbcTemplate;

```
public List<User> getUsers() {  
    return jdbcTemplate.query(  
        "SELECT * FROM users",  
        (rs, rowNum) -> new User(rs.getInt("id"), rs.getString("name"))  
    );  
}
```

Key Idea

- You write **only SQL + logic**
- Spring handles:
 - Connection management
 - Resource cleanup
 - Exception handling

“Spring JDBC is a module that simplifies database operations by reducing boilerplate JDBC code and providing templates like JdbcTemplate for executing SQL queries easily.

Question: Difference between Spring JDBC and Hibernate

Spring JDBC

- Works with **raw SQL queries**
- Developer writes SQL manually
- Uses **JdbcTemplate**
- More control over queries
- Less abstraction

👉 Example:

```
jdbcTemplate.query("SELECT * FROM users", ...);
```

Hibernate

- Uses **ORM (Object Relational Mapping)**
- No need to write SQL (uses HQL/auto-generated SQL)
- Works with **objects instead of tables**
- More abstraction
- Handles mapping automatically

👉 Example:

```
session.save(user);
```

Key Difference

- Spring JDBC → **SQL-based**
 - Hibernate → **Object-based (ORM)**
 - JDBC → more control, more code
 - Hibernate → less code, more abstraction
-

“Spring JDBC requires writing SQL queries manually and provides more control, whereas Hibernate is an ORM framework that maps objects to database tables and reduces the need to write SQL, making development easier.

Question: What is DispatcherServlet?

DispatcherServlet is the **front controller in Spring MVC** that handles all incoming HTTP requests.

What it does

- Receives every request from client
 - Decides **which controller should handle it**
 - Calls the controller method
 - Gets response and returns it to client
-

Flow

Client → DispatcherServlet → Controller → Service → Response

Example

```
@GetMapping("/home")
public String home() {
    return "home";
}
```

- Request /home first goes to **DispatcherServlet**
- Then it forwards to correct controller method

Key Idea

- Central entry point
- Manages entire request flow

“DispatcherServlet is the front controller in Spring MVC that receives all HTTP requests, routes them to appropriate controllers, and returns the response to the client.

Question: How does DispatcherServlet work?

Step-by-step flow

1. Client sends request

/home

2. DispatcherServlet receives request

- Acts as **front controller**

3. Handler Mapping

- Finds **which controller method** should handle request

4. Controller execution

```
@GetMapping("/home")
public String home() {
    return "home";
}
```

- Controller processes request

5. Return value

- Controller returns:
 - View name (HTML/JSP) OR
 - Data (JSON)

6. View Resolver (if view)

- Converts "home" → actual page (home.jsp / HTML)

7. Response sent to client

Simple Flow

Client → DispatcherServlet → HandlerMapping → Controller → ViewResolver → Response

Key Idea

- DispatcherServlet **controls entire request flow**
 - Coordinates all components
-

“DispatcherServlet receives the client request, uses HandlerMapping to find the appropriate controller, executes the controller method, resolves the view using ViewResolver if needed, and returns the response to the client.

Question: What is HandlerInterceptor in Spring MVC?

HandlerInterceptor is used to **intercept HTTP requests before and after they reach the controller.**

- It allows you to add **custom logic** like:
 - Logging
 - Authentication
 - Authorization
-

Methods

1. **preHandle()**
 - Runs **before controller method**
 - Can stop request
 2. **postHandle()**
 - Runs **after controller, before response**
 3. **afterCompletion()**
 - Runs **after response is completed**
-

Example

```
public class MyInterceptor implements HandlerInterceptor {

    public boolean preHandle(HttpServletRequest req, HttpServletResponse res, Object handler) {
        System.out.println("Before Controller");
        return true;
    }

    public void postHandle(...) {
        System.out.println("After Controller");
    }

}
```

Flow

Client → Interceptor (preHandle) → Controller → Interceptor (postHandle) → Response

Key Idea

- Works like a **filter around controller execution**
 - Used for **pre and post processing of requests**
-

“HandlerInterceptor is used in Spring MVC to intercept HTTP requests before and after controller execution, allowing custom logic like logging or authentication to be applied

Question: What is Spring Boot?

Spring Boot is a framework built on top of Spring that **simplifies application development by providing auto-configuration and ready-to-use features.**

Key Idea

- Removes complex configuration
 - Provides **embedded server** (no need to deploy separately)
 - Helps to build applications **quickly**
-

Features

- **Auto-configuration** → configures based on dependencies
 - **Embedded server** → Tomcat (default)
 - **Starter dependencies** → easy setup
 - **Minimal setup (no XML)**
-

Example

```
@SpringBootApplication
public class App {
    public static void main(String[] args) {
        SpringApplication.run(App.class, args);
    }
}
```

Simple Understanding

- Spring → requires more configuration
 - Spring Boot → **“Spring made easy”**
-

“Spring Boot is a framework built on top of Spring that simplifies application development by providing auto-configuration, embedded servers, and minimal setup, allowing developers to build applications quickly.

Question: Difference between Spring Framework and Spring Boot

Spring Framework

- Core framework for building applications
 - Requires **manual configuration** (XML / Java config)
 - No embedded server → need external server (Tomcat)
 - More setup time
 - Gives **more control**
-

Spring Boot

- Built on top of Spring
 - Provides **auto-configuration**
 - Has **embedded server** (Tomcat, Jetty)
 - Uses **starter dependencies**
 - Minimal setup → faster development
-

Key Difference

- Spring → **manual + complex setup**
 - Spring Boot → **auto + simplified setup**
-

“Spring Framework requires manual configuration and setup, whereas Spring Boot simplifies development using auto-configuration, embedded servers, and minimal setup.

Question: Advantages of Spring Boot

Advantages

1. **Auto-Configuration**
 - Automatically configures application based on dependencies
 - Reduces manual setup

2. **Embedded Server**
 - Built-in Tomcat/Jetty
 - No need for external deployment

3. **Starter Dependencies**
 - Predefined dependencies (spring-boot-starter-web, etc.)
 - Easy project setup

4. **Less Boilerplate Code**
 - No XML configuration required
 - Faster development

5. Standalone Application

- Can run using main() method
 - No need for external server
-

6. Production Ready Features

- Provides monitoring, health checks (Actuator)
-

Key Idea

- Faster development
 - Simplified configuration
 - Easy deployment
-

“Spring Boot provides advantages like auto-configuration, embedded servers, starter dependencies, reduced boilerplate code, and standalone application support, making development faster and easier

Question: Disadvantages of Spring Boot

Disadvantages

1. Less Control over Configuration

- Auto-configuration can hide internal details
 - Hard to customize deeply when needed
-

2. Large Application Size

- Includes many dependencies
 - Increases memory usage
-

3. Debugging Difficulty

- Auto-configuration makes it harder to understand internal working
-

4. Startup Time Can Be Higher

- Due to loading many dependencies and auto-configurations
-

5. Not Suitable for Very Small Applications

- Overhead may be unnecessary
-

Key Idea

- Easy to use but sometimes **less transparent and heavier**
-

“Spring Boot reduces configuration but can lead to less control, larger application size, and difficulty in debugging due to auto-configuration.

Question: Can we create a standalone application using Spring Boot?

Yes, **Spring Boot** allows you to create standalone applications.

How

- Comes with **embedded server** (Tomcat by default)
 - No need to deploy on external server
 - Runs using **main() method**
-

Example

```
@SpringBootApplication
public class App {
    public static void main(String[] args) {
        SpringApplication.run(App.class, args);
    }
}
```

Key Idea

- Application runs like a normal Java program
 - No WAR deployment required
-

“Yes, Spring Boot allows creating standalone applications using an embedded server, which can be run directly using a main method without requiring external deployment

Question: What is @SpringBootApplication?

@SpringBootApplication is a **main annotation in Spring Boot** used to **start and configure the application**.

It is a combination of 3 annotations:

1. **@Configuration**
 - Marks class as configuration (can define beans)
 2. **@ComponentScan**
 - Scans packages for components (@Component, @Service, etc.)
 3. **@EnableAutoConfiguration**
 - Automatically configures application based on dependencies
-

Example

```
@SpringBootApplication
public class App {
    public static void main(String[] args) {
        SpringApplication.run(App.class, args);
    }
}
```

Key Idea

- Single annotation → replaces multiple configurations
- Used to **bootstrap (start) Spring Boot app**

“@SpringBootApplication is a combination of @Configuration, @ComponentScan, and @EnableAutoConfiguration used to configure and start a Spring Boot application.”

@EnableAutoConfiguration

- Enables **automatic configuration**
- Configures beans based on dependencies (like DB, MVC, etc.)
- Does **only one job** → auto-config

👉 Example:

@EnableAutoConfiguration

@SpringBootApplication

- **Combination of 3 annotations:**
 - @Configuration
 - @ComponentScan
 - @EnableAutoConfiguration
- Used to **start the entire Spring Boot application**

👉 Example:

@SpringBootApplication

Key Difference

- @EnableAutoConfiguration → only **auto-config**
- @SpringBootApplication → **complete setup + auto-config + scanning**

“@EnableAutoConfiguration is used only to enable automatic configuration, whereas @SpringBootApplication is a combination of @Configuration, @ComponentScan, and @EnableAutoConfiguration used to fully configure and start a Spring Boot application.”

Question: How does Spring Boot internally handle auto-configuration?

Internal Working (Step-by-step)

1. **@SpringBootApplication triggers auto-configuration**
 - It contains @EnableAutoConfiguration

2. **Spring Boot checks classpath (dependencies)**

- Looks at what libraries are present

👉 Example:

- If spring-boot-starter-web → configure MVC
- If MySQL driver → configure DataSource

3. Uses AutoConfiguration classes

- Defined in:

META-INF/spring.factories

- Contains list of configuration classes

4. Conditional Annotations

Spring uses conditions like:

- @ConditionalOnClass → if class exists
- @ConditionalOnMissingBean → if bean not already defined
- @ConditionalOnProperty → based on properties

👉 Example:

@ConditionalOnClass(DataSource.class)

5. Creates Beans Automatically

- Based on conditions, Spring Boot **creates required beans**

Example Flow

Dependency → Condition Check → AutoConfig Class → Bean Creation

Key Idea

- No manual config
- Spring Boot **decides automatically based on environment**

“Spring Boot handles auto-configuration by using @EnableAutoConfiguration, which scans predefined configuration classes from spring.factories, checks conditions based on classpath, properties, and existing beans, and automatically creates required beans.

Question: What is spring.factories file?

spring.factories is a **configuration file used by Spring Boot to load auto-configuration classes.**

Location

META-INF/spring.factories

What it contains

- List of **auto-configuration classes**
- Key used:

org.springframework.boot.autoconfigure.EnableAutoConfiguration=\ncom.example.MyAutoConfig,\ncom.example.DBConfig

How it works

1. Spring Boot reads spring.factories
2. Gets list of configuration classes
3. Loads them into application context
4. Applies conditions → creates beans automatically

Key Idea

- Acts like a **registry of auto-config classes**
- Helps Spring Boot decide **what to configure automatically**

“spring.factories is a configuration file located in META-INF that contains a list of auto-configuration classes, which Spring Boot loads to perform automatic configuration of the application

Question: How do you exclude auto-configuration?

You can exclude auto-configuration in Spring Boot in 3 ways:

1. Using @SpringBootApplication

@SpringBootApplication(exclude = DataSourceAutoConfiguration.class)

```
public class App {  
}
```

- Excludes specific auto-configuration class

2. Using @EnableAutoConfiguration

@EnableAutoConfiguration(exclude = DataSourceAutoConfiguration.class)

3. Using application.properties

spring.autoconfigure.exclude=org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration

Key Idea

- Prevents Spring Boot from automatically configuring unwanted components

“Auto-configuration can be excluded using @SpringBootApplication or @EnableAutoConfiguration with the exclude attribute, or by specifying classes in application.properties using spring.autoconfigure.exclude.

Question: What is Spring Boot Starter?

Spring Boot Starter is a **predefined set of dependencies** that makes it easy to add specific functionality to a project.

Why it is used

- Avoid adding multiple dependencies manually
 - Provides **ready-to-use configuration**
 - Simplifies project setup
-

Example

<dependency>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-web</artifactId>

</dependency>

- This single starter includes:
 - Spring MVC
 - Embedded Tomcat
 - Jackson (JSON)
-

Common Starters

- spring-boot-starter-web → Web applications
 - spring-boot-starter-data-jpa → Database (JPA)
 - spring-boot-starter-security → Security
-

Key Idea

- One dependency → multiple related libraries
-

“Spring Boot Starter is a predefined dependency that bundles multiple related libraries together to simplify configuration and quickly add specific functionality to a Spring Boot application.”

Question: What is spring-boot-starter-data-jpa?

spring-boot-starter-data-jpa is a Spring Boot starter used for **database operations using JPA (Java Persistence API)**.

What it provides

- **Spring Data JPA** (repository support)
 - **Hibernate** (ORM implementation)
 - **JPA APIs**
 - Transaction support
-

What it does

- Allows working with **Java objects instead of SQL**

- Provides **ready repository interfaces** like JpaRepository
 - Automatically handles **CRUD operations**
-

Example

```
public interface UserRepository extends JpaRepository<User, Integer> {  
}
```

- No need to write SQL for basic operations
-

Important

- Does NOT include database driver
 - You must add (MySQL, PostgreSQL, etc.)
-

“spring-boot-starter-data-jpa is a starter that provides JPA and Hibernate support for database operations, allowing developers to perform CRUD operations using repository interfaces without writing SQL.”

Question: How does Spring Boot handle dependency management?

Spring Boot handles dependency management using **Maven/Gradle with a parent configuration that provides predefined versions of dependencies.**

How it works

1. Spring Boot Parent (Maven)

```
<parent>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-parent</artifactId>  
</parent>
```

- Defines **compatible versions** of all dependencies
 - You don't need to specify versions manually
-

2. Starter Dependencies

```
<artifactId>spring-boot-starter-web</artifactId>
```

- Brings multiple dependencies together
 - All versions are **managed automatically**
-

3. Dependency Management (BOM)

- Spring Boot uses a **BOM (Bill of Materials)**
 - Ensures all libraries work together without conflicts
-

Example

```
<dependency>  
  <groupId>org.springframework.boot</groupId>
```

```
<artifactId>spring-boot-starter-web</artifactId>  
</dependency>
```

- No version needed → Spring Boot handles it

Key Idea

- No version conflicts
- No manual version management
- Everything is **pre-tested and compatible**

“Spring Boot handles dependency management using a parent POM and BOM, which provide predefined compatible versions of dependencies, so developers don’t need to specify versions manually

Question: What is application.properties?

application.properties is a **configuration file in Spring Boot** used to define **application settings**.

What it is used for

- Configure:
 - Server port
 - Database connection
 - Logging settings
 - Custom properties

Example

server.port=8081

spring.datasource.url=jdbc:mysql://localhost:3306/mydb

spring.datasource.username=root

spring.datasource.password=1234

Key Idea

- Uses **key = value format**
- Central place for configuration

“application.properties is a configuration file in Spring Boot used to define application settings like server port, database configuration, and other properties using key-value pairs.

Question: What is application.yml?

application.yml is a **configuration file in Spring Boot** used to define **application settings in YAML format**.

What it is used for

- Same purpose as application.properties
 - Used for:
 - Server config
 - Database config
 - Custom properties
-

Example

server:

port: 8081

spring:

datasource:

url: jdbc:mysql://localhost:3306/mydb

username: root

password: 1234

Key Idea

- Uses **hierarchical structure (nested format)**
 - More **readable for complex configurations**
-

“application.yml is a Spring Boot configuration file that uses YAML format to define application settings in a hierarchical and more readable structure compared to properties files.”

Question: Difference between application.properties and application.yml

application.properties

- Uses **key = value format**
- Flat structure
- Less readable for complex configurations

👉 Example:

server.port=8081

spring.datasource.url=jdbc:mysql://localhost:3306/mydb

spring.datasource.username=root

spring.datasource.password=1234

application.yml

- Uses **YAML (hierarchical format)**

- Supports nesting (clean structure)
- More readable for complex configs

👉 Example:

server:

port: 8081

spring:

datasource:

url: jdbc:mysql://localhost:3306/mydb

username: root

password: 1234

Key Difference

- properties → **flat, simple**
- yaml → **hierarchical, readable**

“application.properties uses a flat key-value format, whereas application.yml uses a hierarchical YAML format that is more readable and suitable for complex configurations.”

Question: How to change the default server port?

Using application.properties

server.port=9090

Using application.yml

server:

port: 9090

Using command line

java -jar app.jar --server.port=9090

Key Idea

- Default port = **8080**
 - You can override it using configuration
-

“The default server port in Spring Boot can be changed by setting server.port in application.properties or application.yml, or by passing it as a command-line argument

Question: How does Spring Boot handle embedded servers?

Spring Boot provides **embedded servers** (like Tomcat) and manages them automatically.

How it works

1. Starter dependency

`<artifactId>spring-boot-starter-web</artifactId>`

- Includes **embedded Tomcat (default)**
-

2. Application starts

`SpringApplication.run(App.class, args);`

- Spring Boot:
 - Creates application context
 - Starts embedded server internally
-

3. Server runs inside application

- No need to deploy WAR file
 - Runs as **standalone JAR**
-

Supported Servers

- Tomcat (default)
 - Jetty
 - Undertow
-

Key Idea

- Server is part of application
 - Auto-configured and started automatically
-

“Spring Boot handles embedded servers by including them through starter dependencies and automatically starting them when the application runs, allowing applications to run as standalone without external deployment.”

Question: What are other servers supported by Spring Boot?

Supported Embedded Servers

1. Tomcat (Default)

- Comes by default with spring-boot-starter-web
-

2. Jetty

- Lightweight alternative to Tomcat
-

3. Undertow

- High-performance, non-blocking server

How to use other servers

👉 Exclude Tomcat and add another server:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
  <exclusions>
    <exclusion>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-tomcat</artifactId>
    </exclusion>
  </exclusions>
</dependency>
```

Then add Jetty or Undertow:

```
<artifactId>spring-boot-starter-jetty</artifactId>
```

Key Idea

- Default → Tomcat
- Can replace with Jetty or Undertow easily

“Spring Boot supports embedded servers like Tomcat (default), Jetty, and Undertow, and allows switching between them by excluding Tomcat and adding the desired server dependency.”

Question: What are Spring Profiles?

Spring Profiles are used to **manage different configurations for different environments** like development, testing, and production.

Why needed

- Different environments need different settings
 - Dev → local DB
 - Prod → real DB

Example

application-dev.properties

server.port=8081

application-prod.properties

server.port=8080

Activate profile

spring.profiles.active=dev

Using @Profile

@Profile("dev")

@Component

class DevService {}

- Bean loads only in **dev profile**
-

Key Idea

- Separate config per environment
 - Avoid changing code manually
-

“Spring Profiles are used to define and manage environment-specific configurations, allowing different settings for development, testing, and production environments.”

Question: What is REST API?

REST API (Representational State Transfer) is a way to **build web services using HTTP protocols**, where data is treated as **resources** and accessed using standard methods.

Key Idea

- Everything is a **resource** (user, product, etc.)
 - Uses HTTP methods to perform operations
-

HTTP Methods

- **GET** → Read data
 - **POST** → Create data
 - **PUT** → Update data
 - **DELETE** → Delete data
-

Example

GET /users → get all users

POST /users → create user

PUT /users/1 → update user

DELETE /users/1 → delete user

Important Feature

- **Stateless** → each request is independent (no server memory of previous request)
-

“REST API is an architectural style for building web services that uses HTTP methods to perform operations on resources, following principles like statelessness and standard communication.”

Question: Difference between REST and SOAP

REST

- **Architectural style**
 - Uses **HTTP methods** (GET, POST, PUT, DELETE)
 - Data format: **JSON (main), XML**
 - **Lightweight and fast**
 - Stateless
-

SOAP

- **Protocol**
 - Uses **XML only**
 - Follows strict standards (WSDL, etc.)
 - **Heavier and slower**
 - Supports built-in security and transactions
-

Key Difference

- REST → **simple, fast, flexible**
 - SOAP → **strict, secure, heavy**
-

“REST is an architectural style that uses HTTP methods and supports lightweight data formats like JSON, whereas SOAP is a protocol that uses XML with strict standards and is heavier but provides built-in security features.”

Question: Difference between @Controller and @RestController

@Controller

- Used for **web applications (MVC)**
- Returns **view (HTML/JSP)**
- Needs @ResponseBody to return data

👉 Example:

@Controller

```
class MyController {  
  
    @GetMapping("/home")  
    public String home() {  
        return "home"; // returns view  
    }  
}
```

@RestController

- Used for **REST APIs**

- Returns **data (JSON/XML)**
- No need for `@ResponseBody`

👉 Example:

```
@RestController
class MyRestController {

    @GetMapping("/data")
    public String data() {
        return "Hello"; // returns JSON
    }
}
```

Key Difference

- `@Controller` → **returns view**
- `@RestController` → **returns data**

“@Controller is used to return views in Spring MVC, whereas @RestController is used to build REST APIs and return data directly in JSON or XML format.”

Question: What is the default return type in @RestController?

In `@RestController`, the default return type is **JSON**.

Why

- `@RestController` = `@Controller` + `@ResponseBody`
- So return value is automatically converted to **JSON response**

Example

```
@RestController
class UserController {

    @GetMapping("/user")
    public User getUser() {
        return new User(1, "Suhas");
    }
}
```

👉 Output (JSON):

```
{
  "id": 1,
  "name": "Suhas"
}
```

“The default return type in @RestController is JSON, as it automatically converts return values into JSON responses using @ResponseBody.”

Question: What is @RequestMapping?

@RequestMapping is used to **map HTTP requests to controller methods**.

What it does

- Defines **URL path**
 - Can specify **HTTP method** (GET, POST, etc.)
 - Used at **class level or method level**
-

Example

```
@RestController
@RequestMapping("/api")
class MyController {

    @RequestMapping(value = "/users", method = RequestMethod.GET)
    public String getUsers() {
        return "Users list";
    }
}
```

- URL: /api/users
- Method: GET

Shortcut Annotations

- @GetMapping → GET
 - @PostMapping → POST
 - @PutMapping → PUT
 - @DeleteMapping → DELETE
-

Key Idea

- Maps URL + HTTP method → controller method
-

“@RequestMapping is used to map HTTP requests to controller methods by specifying the URL path and HTTP method.”

Question: What is @PathVariable?

@PathVariable is used to **extract values from the URL path and pass them to a method parameter**.

Example

```
@RestController
class UserController {

    @GetMapping("/users/{id}")
    public String getUser(@PathVariable int id) {
```

```
    return "User id: " + id;
}
}
```

👉 Request:

/users/10

👉 Output:

User id: 10

Key Idea

- Used for **dynamic values in URL**
- Value is taken from {} in path

“@PathVariable is used to extract values from the URL path and bind them to method parameters in a controller.”

Question: What is returned by @PathVariable?

@PathVariable itself **does not return anything**.

- It **extracts a value from the URL path**
- Passes that value to the **method parameter**

Example

```
@GetMapping("/users/{id}")
public String getUser(@PathVariable int id) {
    return "User id: " + id;
}
```

👉 Here:

- /users/10 → id = 10
- Method returns → "User id: 10"

Key Idea

- @PathVariable → **input (from URL)**
- Method → **returns response**

“@PathVariable does not return anything; it extracts a value from the URL and binds it to a method parameter.”

Question: What is @RequestParam?

@RequestParam is used to **extract query parameters from the URL and bind them to method parameters**.

Example


```
@RestController
class UserController {

    @GetMapping("/users")
    public String getUser(@RequestParam String name) {
        return "User: " + name;
    }
}
```

👉 Request:

/users?name=Suhas

👉 Output:

User: Suhas

Key Idea

- Used for **query parameters (?key=value)**
- Value is taken from URL after ?

“@RequestParam is used to extract query parameters from the URL and bind them to method parameters in a controller.”

Question: What is @RequestBody?

@RequestBody is used to **read data from the HTTP request body and convert it into a Java object.**

Example

```
@RestController
class UserController {

    @PostMapping("/users")
    public String createUser(@RequestBody User user) {
        return user.getName();
    }
}
```

👉 Request (JSON):

```
{
  "id": 1,
  "name": "Suhas"
}
```

👉 Spring converts JSON → User object automatically

Key Idea

- Used in **POST/PUT requests**
- Converts **JSON → Java object**

“@RequestBody is used to bind the HTTP request body to a Java object by automatically converting JSON or XML data into the object.”

Question: What are consumes and produces?

consumes and produces are used in Spring to **define the type of data a REST API can accept and return.**

consumes

- Specifies **what type of request body the API accepts**
- Example: JSON, XML

@PostMapping(value = "/users", consumes = "application/json")

👉 Means:

- API accepts **JSON input only**
-

produces

- Specifies **what type of response the API returns**

@GetMapping(value = "/users", produces = "application/json")

👉 Means:

- API returns **JSON output**
-

Example

```
@PostMapping(  
    value = "/users",  
    consumes = "application/json",  
    produces = "application/json"  
)  
public User createUser(@RequestBody User user) {  
    return user;  
}
```

Key Idea

- consumes → **input format**
 - produces → **output format**
-

“consumes defines the media type that the API can accept in the request, while produces defines the media type that the API returns in the response.”

Question: How do you return XML from a REST API?

To return XML in Spring Boot, you need to:

1. Add Jackson XML dependency

```
<dependency>
  <groupId>com.fasterxml.jackson.dataformat</groupId>
  <artifactId>jackson-dataformat-xml</artifactId>
</dependency>
```

2. Set produces = "application/xml"

```
@RestController
class UserController {

    @GetMapping(value = "/user", produces = "application/xml")
    public User getUser() {
        return new User(1, "Suhas");
    }
}
```

3. (Optional) Add XML annotations

```
@JacksonXmlRootElement(localName = "user")
class User {
    public int id;
    public String name;
}
```

Output (XML)

```
<user>
  <id>1</id>
  <name>Suhas</name>
</user>
```

Key Idea

- Add XML support
 - Set produces = "application/xml"
-

“To return XML from a REST API, add Jackson XML dependency and set produces = 'application/xml' in the controller, so Spring converts the response object into XML format.”

Question: Why return response body instead of String?

Returning String

```
@GetMapping("/data")
public String data() {
    return "home";
}
```

- In @Controller → treated as **view name (HTML/JSP)**
 - Not actual data
 - Used for **web pages**
-

Returning Response Body (Object / JSON)

```
@RestController
@GetMapping("/data")
public User data() {
    return new User(1, "Suhas");
}
```

- Returned as **JSON response**
 - Used for **REST APIs**
 - Automatically converted using @ResponseBody
-

Why use response body

- To send **actual data (JSON/XML)** to client
 - Not view names
 - Required for APIs (frontend, mobile apps)
-

Key Idea

- String → **view (MVC)**
 - Response body → **data (API)**
-

“We return response body instead of String to send actual data like JSON or XML to the client, whereas returning String in a controller is treated as a view name.”

Question: How do you expose and consume REST APIs in Spring Boot?

1. Expose REST API (Create API)

Use @RestController and mapping annotations:

```
@RestController
@RequestMapping("/api")
class UserController {
```

```
@GetMapping("/users")
public List<User> getUsers() {
    return List.of(new User(1, "Suhas"));
}

@PostMapping("/users")
public User createUser(@RequestBody User user) {
    return user;
}
}
```

- @RestController → makes it REST API
- @GetMapping, @PostMapping → map HTTP methods
- @RequestBody → read input

2. Consume REST API (Call API)

Using RestTemplate

```
RestTemplate restTemplate = new RestTemplate();
```

```
User user = restTemplate.getForObject(
    "http://localhost:8080/api/users/1",
    User.class
);
```

Using WebClient (Modern way)

```
WebClient webClient = WebClient.create();
```

```
User user = webClient.get()
    .uri("http://localhost:8080/api/users/1")
    .retrieve()
    .bodyToMono(User.class)
    .block();
```

Key Idea

- Expose → create API using controller
 - Consume → call API using RestTemplate or WebClient
-

“REST APIs are exposed in Spring Boot using @RestController and mapping annotations, and consumed using clients like RestTemplate or WebClient to make HTTP calls.

Question: How do you interact with third-party APIs in Spring Boot?

You interact with third-party APIs by making **HTTP calls** using clients like **RestTemplate** or **WebClient**.

1. Using RestTemplate (Traditional)

```
RestTemplate restTemplate = new RestTemplate();
```

```
String response = restTemplate.getForObject(  
    "https://api.example.com/data",  
    String.class  
);
```

- Simple and synchronous
 - Blocks until response is received
-

2. Using WebClient (Recommended)

```
WebClient webClient = WebClient.create();
```

```
String response = webClient.get()  
    .uri("https://api.example.com/data")  
    .retrieve()  
    .bodyToMono(String.class)  
    .block();
```

- Modern and non-blocking
 - Better for scalable applications
-

For POST request

```
webClient.post()  
    .uri("https://api.example.com/data")  
    .bodyValue(requestObject)  
    .retrieve()  
    .bodyToMono(String.class)  
    .block();
```

Key Idea

- Use HTTP client to call external APIs
 - Send request → receive response
-

“In Spring Boot, third-party APIs are consumed using HTTP clients like RestTemplate or WebClient, which send requests and receive responses from external services

Question: How do you handle exceptions in Spring Boot?

Spring Boot handles exceptions using **@ExceptionHandler** and **@ControllerAdvice** (global handling).

1. Using @ExceptionHandler (inside controller)

@RestController

class UserController {

 @ExceptionHandler(RuntimeException.class)

 public String handleException(RuntimeException ex) {

 return "Error: " + ex.getMessage();

 }

}

- Handles specific exceptions in that controller
-

2. Global Exception Handling (@ControllerAdvice)

@ControllerAdvice

class GlobalExceptionHandler {

 @ExceptionHandler(RuntimeException.class)

 public String handleGlobalException(RuntimeException ex) {

 return "Global Error: " + ex.getMessage();

 }

}

- Handles exceptions across all controllers
 - Avoids repeating code
-

Flow

Request → Controller → Exception → Handler → Response

Key Idea

- Centralized error handling
 - Returns meaningful responses instead of crashes
-

“Exceptions in Spring Boot are handled using @ExceptionHandler for specific cases and @ControllerAdvice for global handling, allowing centralized and clean error management.”

Question: What is @ExceptionHandler?

@ExceptionHandler is used to **handle specific exceptions in Spring controllers**.

- It catches exceptions thrown by controller methods
- Returns a **custom response instead of default error**

Example

```
@RestController
class UserController {

    @GetMapping("/user")
    public String getUser() {
        throw new RuntimeException("Error occurred");
    }

    @ExceptionHandler(RuntimeException.class)
    public String handleException(RuntimeException ex) {
        return "Custom Error: " + ex.getMessage();
    }
}
```

How it works

- Exception occurs in method
- Spring calls the method annotated with @ExceptionHandler
- Returns custom response

Key Idea

- Handles **specific exceptions locally (inside controller)**

“@ExceptionHandler is used to handle specific exceptions in a controller by catching them and returning a custom response instead of the default error.”

Question: What is @ControllerAdvice?

@ControllerAdvice is used for **global exception handling across all controllers**.

- Instead of writing @ExceptionHandler in every controller
- You write it **once in a single class**
- Applies to the entire application

Example

```
@ControllerAdvice
class GlobalExceptionHandler {

    @ExceptionHandler(RuntimeException.class)
```



```
public String handleException(RuntimeException ex) {  
    return "Global Error: " + ex.getMessage();  
}  
}
```

How it works

- Any controller throws exception
 - Spring redirects it to this class
 - Returns custom response
-

Key Idea

- Centralized exception handling
 - Avoids duplicate code
-

“@ControllerAdvice is used to handle exceptions globally across all controllers by defining centralized exception handling logic.”

Question: How does Spring Boot handle global exception handling?

Spring Boot handles global exceptions using **@ControllerAdvice + @ExceptionHandler**.

How it works (flow)

1. **Request comes to controller**
 2. **Exception occurs in any method**
 3. Spring looks for a matching **@ExceptionHandler**
 4. If not found in controller → goes to **@ControllerAdvice**
 5. Global handler method executes
 6. **Custom response is returned to client**
-

Example

```
@ControllerAdvice  
class GlobalExceptionHandler {  
  
    @ExceptionHandler(RuntimeException.class)  
    public String handleException(RuntimeException ex) {  
        return "Error: " + ex.getMessage();  
    }  
}
```

Key Idea

- One place → handles exceptions for all controllers
 - Cleaner and reusable error handling
-

“Spring Boot handles global exception handling using @ControllerAdvice with @ExceptionHandler methods, which catch exceptions across all controllers and return custom responses.”

Question: What is Spring Data JPA?

Spring Data JPA is a module of Spring that **simplifies database operations using JPA (Java Persistence API)**.

What it does

- Eliminates **boilerplate code**
 - Provides **ready-made repository interfaces**
 - No need to write SQL for basic operations
-

Example

```
public interface UserRepository extends JpaRepository<User, Integer> {  
}
```

- Gives methods like:
 - save()
 - findById()
 - findAll()
 - deleteById()
-

Key Idea

- Work with **Java objects instead of SQL**
 - Uses Hibernate internally
-

“Spring Data JPA is a Spring module that simplifies database operations by providing repository interfaces and eliminating the need to write boilerplate SQL code using JPA.”

Question: Difference between CrudRepository and JpaRepository

CrudRepository

- Provides **basic CRUD operations**
- Methods:
 - save()
 - findById()
 - findAll()
 - deleteById()
- Limited functionality

👉 Example:

```
public interface UserRepo extends CrudRepository<User, Integer> {  
}
```

JpaRepository

- Extends **CrudRepository + PagingAndSortingRepository**
- Provides **advanced features**:
 - Pagination
 - Sorting
 - JPA-specific methods (flush(), saveAndFlush())

👉 Example:

```
public interface UserRepo extends JpaRepository<User, Integer> {  
}
```

Key Difference

- CrudRepository → **basic operations only**
 - JpaRepository → **advanced + pagination + sorting + JPA features**
-

“CrudRepository provides basic CRUD operations, whereas JpaRepository extends it and provides advanced features like pagination, sorting, and JPA-specific methods.”

Question: Difference between Hibernate and JPA

JPA (Java Persistence API)

- **Specification (set of rules/interfaces)**
- Defines **how ORM should work**
- Does **not provide implementation**

👉 Example:

```
EntityManager em;
```

Hibernate

- **Framework (implementation of JPA)**
- Provides actual functionality for ORM
- Handles mapping, queries, caching

👉 Example:

```
session.save(user);
```

Key Difference

- JPA → **interface/rules (no implementation)**
 - Hibernate → **actual implementation (does the work)**
-

Simple Understanding

- JPA = **what to do**
 - Hibernate = **how it is done**
-

“JPA is a specification that defines how ORM should be implemented, whereas Hibernate is a framework that implements JPA and provides actual database operations

Question: Advantages of Hibernate

Advantages

1. No need to write SQL

- Uses HQL / automatic SQL generation
 - Reduces boilerplate code
-

2. ORM (Object Relational Mapping)

- Maps Java objects ↔ database tables
 - Work with objects instead of queries
-

3. Database Independence

- Same code works with different databases
 - Just change configuration
-

4. Caching Support

- Improves performance
 - Reduces database hits
-

5. Automatic Table Creation

- Can create/update tables automatically
-

6. Transaction Management

- Supports transactions easily
-

Key Idea

- Less code
 - Easy database handling
 - Better performance
-

“Hibernate simplifies database operations using ORM, reduces the need to write SQL, provides caching and database independence, and improves performance and maintainability.

Question: What is lazy loading?

Lazy loading is a concept where **data is loaded only when it is actually needed**, not at the time of initial fetch.

Example

```
User user = entityManager.find(User.class, 1);
```

- Only **User data** is loaded
- Related data (like Orders) is **NOT loaded immediately**

👉 When accessed:

```
user.getOrders();
```

- Now orders are fetched from DB
-

Key Idea

- Load data **on demand**
 - Improves performance
-

Opposite

- **Eager loading** → loads everything immediately
-

“Lazy loading is a technique where related data is fetched only when it is accessed, rather than loading it at the initial query, improving performance.”

Question: What is caching in Hibernate?

Caching in Hibernate is used to **store frequently accessed data in memory** so that repeated database queries can be avoided.

Why it is used

- Reduces **database hits**
 - Improves **performance**
 - Faster data access
-

Types of Cache

1. First-Level Cache (Session Cache)

- Enabled by default
- Works within a **single session**

👉 Example:

```
User user1 = session.get(User.class, 1);
```

```
User user2 = session.get(User.class, 1);
```

- Second call does NOT hit DB
 - Data comes from cache
-

2. Second-Level Cache

- Works across **multiple sessions**
 - Needs configuration (EhCache, etc.)
-

Key Idea

- Store data in memory → avoid repeated DB calls
-

“Caching in Hibernate stores frequently accessed data in memory to reduce database access and improve performance, with first-level cache being default per session and second-level cache shared across sessions.”

Question: How do you configure multiple databases in Spring Boot?

To configure multiple databases, you need to **define separate configurations for each database.**

1. Define properties (application.properties)

```
spring.datasource.db1.url=jdbc:mysql://localhost:3306/db1
spring.datasource.db1.username=root
spring.datasource.db1.password=1234
```

```
spring.datasource.db2.url=jdbc:mysql://localhost:3306/db2
spring.datasource.db2.username=root
spring.datasource.db2.password=1234
```

2. Create DataSource beans

```
@Configuration
class DataSourceConfig {

    @Bean
    @Primary
    public DataSource db1DataSource() {
        return DataSourceBuilder.create().build();
    }

    @Bean
    public DataSource db2DataSource() {
        return DataSourceBuilder.create().build();
    }
}
```

3. Configure EntityManager & TransactionManager

```
@Configuration
@EnableJpaRepositories(
    basePackages = "com.app.repo1",
```

```
entityManagerFactoryRef = "em1",
transactionManagerRef = "tx1"
)
class DB1Config{ }
    • Repeat for second database with different package
```

Key Idea

- Separate:
 - DataSource
 - EntityManager
 - Repository packages
-

“Multiple databases in Spring Boot are configured by defining separate DataSource, EntityManager, and TransactionManager beans for each database and mapping them to different repository packages.”

Question: What is Spring Security?

Spring Security is a framework used to **secure Spring applications** by providing **authentication and authorization**.

What it does

- **Authentication** → verifies who the user is (login)
 - **Authorization** → decides what the user can access
-

Example

- User logs in with username & password → authentication
 - Admin can delete data, user cannot → authorization
-

Features

- Login & logout support
 - Password encryption (BCrypt)
 - Role-based access control
 - Protection against attacks (CSRF, etc.)
-

Key Idea

- Controls **who can access what**
-

“Spring Security is a framework used to secure applications by providing authentication and authorization, ensuring that only authorized users can access specific resources.”

Question: How does Spring Security work?

Step-by-step flow

1. **Client sends request (login or API call)**

- Example: username & password
-

2. **Request goes through Security Filters**

- Spring Security uses a **filter chain**
 - Every request is intercepted before reaching controller
-

3. **Authentication process**

- UsernamePasswordAuthenticationFilter extracts credentials
 - Passes to **AuthenticationManager**
-

4. **AuthenticationManager**

- Delegates to **AuthenticationProvider**
-

5. **AuthenticationProvider**

- Calls **UserDetailsService**
-

6. **UserDetailsService**

- Loads user from DB using:

loadUserByUsername(username)

7. **Password verification**

- Uses **PasswordEncoder (BCrypt)**
 - Compares entered password with stored password
-

8. **If valid**

- Authentication successful
 - User details stored in **SecurityContext (session created)**
-

9. **Request proceeds to controller**

For next requests

- Client sends **JSESSIONID**
 - Spring loads user from session → no need to login again
-

Simple Flow

Client → Filter → AuthenticationManager → UserDetailsService → DB → Success → Controller

Key Idea

- Uses **filter chain + authentication flow**
- Validates user before allowing access

“Spring Security works by intercepting requests using a filter chain, authenticating users through AuthenticationManager and UserDetailsService, validating credentials, and storing authentication details in the SecurityContext before allowing access to resources.”

Question: Explain authentication and authorization

Authentication

- **Authentication = Who are you?**
- Verifies **user identity**

👉 Example:

- Login with username & password
- System checks if user is valid

✓ If correct → user is authenticated

✗ If wrong → access denied

Authorization

- **Authorization = What can you access?**
- Happens **after authentication**

👉 Example:

- Admin → can delete data
- User → can only view data

✓ Checks user **roles/permissions**

Key Difference

- Authentication → **verify identity**
- Authorization → **verify permissions**

Simple Flow

Login → Authentication → Authorization → Access

“Authentication verifies the identity of a user, while authorization determines what actions or resources the authenticated user is allowed to access.”

Question: How can you secure REST APIs in Spring Boot?

You secure REST APIs using **Spring Security**.

1. Add dependency

```
<artifactId>spring-boot-starter-security</artifactId>
```

2. Configure security (SecurityFilterChain)

@Bean

```
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {  
    http  
        .csrf().disable()  
        .authorizeHttpRequests(auth -> auth  
            .requestMatchers("/public/**").permitAll()  
            .anyRequest().authenticated()  
        )  
        .httpBasic();  
    return http.build();  
}
```

3. Authentication methods

- **Basic Authentication**
 - **JWT (Token-based authentication)**
 - OAuth2 (advanced)
-

4. Password encryption

@Bean

```
public PasswordEncoder passwordEncoder() {  
    return new BCryptPasswordEncoder();  
}
```

5. Role-based access

```
@PreAuthorize("hasRole('ADMIN')")  
public void deleteUser() {}
```

Key Idea

- Authenticate user
 - Authorize access
 - Protect endpoints
-

“REST APIs in Spring Boot are secured using Spring Security by configuring authentication, authorization, password encoding, and optionally token-based mechanisms like JWT.”

Question: How do you enable CORS in Spring MVC?

CORS (Cross-Origin Resource Sharing) is enabled to allow requests from **different domains**.

1. Using @CrossOrigin (simple way)

@RestController

@CrossOrigin(origins = "http://localhost:3000")

class UserController {

 @GetMapping("/users")

 public String getUsers() {

 return "Users";

 }

}

- Allows requests from specified origin
-

2. Global CORS configuration

@Configuration

class WebConfig implements WebMvcConfigurer {

 @Override

 public void addCorsMappings(CorsRegistry registry) {

 registry.addMapping("/**")

 .allowedOrigins("http://localhost:3000")

 .allowedMethods("GET", "POST", "PUT", "DELETE");

 }

}

- Applies to all controllers
-

Key Idea

- @CrossOrigin → per controller
 - WebMvcConfigurer → global
-

“CORS in Spring MVC can be enabled using @CrossOrigin at the controller level or by configuring it globally using WebMvcConfigurer to allow cross-origin requests.”

Question: What is Spring Cloud?

Spring Cloud is a framework used to **build and manage microservices-based applications**.

What it provides

- Tools for handling **distributed systems**
 - Helps manage communication between microservices
-

Features

- **Service Discovery** (Eureka)
 - **Load Balancing**
 - **API Gateway**
 - **Configuration Management**
 - **Circuit Breaker (fault tolerance)**
-

Example idea

- Multiple services:
 - User Service
 - Order Service
 - Payment Service

Spring Cloud helps them **communicate and work together smoothly**

Key Idea

- Used in **microservices architecture**
 - Handles **complex distributed system problems**
-

“Spring Cloud is a framework used to build and manage microservices applications by providing tools for service discovery, load balancing, configuration management, and fault tolerance.”

Question: What is microservices architecture?

Microservices architecture is a design approach where an application is **divided into small, independent services**, each handling a specific functionality.

Key Idea

- Instead of one big application (monolith)
 - Build multiple **small services**
-

Example

- User Service → handles users
- Order Service → handles orders
- Payment Service → handles payments

Each service:

- Runs independently

- Has its own logic (can have its own DB)
-

Characteristics

- **Loosely coupled**
 - **Independently deployable**
 - Communicate via **REST APIs**
-

Benefits

- Easy to scale
 - Faster development
 - Better fault isolation
-

Simple Understanding

Monolith → one big system

Microservices → many small services

“Microservices architecture is a design approach where an application is divided into small, independent services that communicate with each other, allowing scalability, flexibility, and independent deployment.”

Question: Difference between monolithic and microservices architecture

Monolithic Architecture

- Entire application is built as **one single unit**
- All modules (UI, business logic, DB) are tightly connected
- Single deployment

👉 Example:

- One application handles users, orders, payments
-

Microservices Architecture

- Application is divided into **multiple small services**
- Each service handles a **specific functionality**
- Each service can be deployed independently

👉 Example:

- User Service
 - Order Service
 - Payment Service
-

Key Differences

- Monolithic → **single large application**
- Microservices → **multiple small independent services**
- Monolithic → tightly coupled

- Microservices → loosely coupled
- Monolithic → hard to scale
- Microservices → easy to scale
- Monolithic → single deployment
- Microservices → independent deployment

“Monolithic architecture is a single, tightly coupled application, whereas microservices architecture consists of multiple loosely coupled services that can be developed, deployed, and scaled independently.”

Question: Advantages and disadvantages of microservices

Advantages

1. **Independent Deployment**
 - Each service can be deployed separately
2. **Scalability**
 - Scale only required services, not entire app
3. **Fault Isolation**
 - Failure in one service doesn't affect others
4. **Technology Flexibility**
 - Different services can use different technologies
5. **Faster Development**
 - Teams can work on services independently

Disadvantages

1. **Complexity**
 - Managing multiple services is difficult
2. **Communication Overhead**
 - Services communicate over network (REST, etc.)
3. **Deployment Complexity**
 - Requires tools (Docker, Kubernetes, etc.)
4. **Data Management Issues**
 - Multiple databases → consistency challenges
5. **Debugging Difficulty**
 - Hard to trace issues across services

Key Idea

- Powerful but adds **complexity**

“Microservices provide advantages like scalability, flexibility, and independent deployment, but they introduce complexity in communication, deployment, and data management.”

Question: What is load balancing?

Load balancing is the process of **distributing incoming requests across multiple servers** to ensure no single server is overloaded.

Simple understanding

- Instead of sending all requests to one server
 - Requests are **shared among multiple servers**
-

Example

User requests → Server1, Server2, Server3

- Request 1 → Server1
 - Request 2 → Server2
 - Request 3 → Server3
-

Why it is used

- Improves **performance**
 - Increases **availability**
 - Prevents **server overload**
-

Types

- **Round Robin** → requests distributed one by one
 - **Least Connections** → send to server with fewer connections
-

Key Idea

- Balance load → better performance & reliability
-

“Load balancing is the process of distributing incoming requests across multiple servers to improve performance, scalability, and availability.”

Question: What is Circuit Breaker?

Circuit Breaker is a pattern used to **prevent a system from repeatedly calling a failing service**, improving fault tolerance in microservices.

Simple understanding

- If a service is **failing repeatedly**
 - Circuit Breaker **stops calling it temporarily**
-

States

1. **Closed**
 - Normal state
 - Requests go to service
2. **Open**

- Service is failing
 - Requests are **blocked** (fail fast)
- ### 3. Half-Open
- Allows limited requests
 - Checks if service is recovered
-

Example

- Payment service is down
 - Instead of trying again and again → Circuit Breaker:
 - Stops calls
 - Returns fallback response
-

Key Idea

- Avoids cascading failures
 - Improves system stability
-

“Circuit Breaker is a design pattern that stops calls to a failing service after repeated failures and allows recovery checks, improving fault tolerance in distributed systems.”

Question: How to implement Circuit Breaker in Spring Boot?

Spring Boot uses **Resilience4j** to implement Circuit Breaker.

1. Add dependency

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-circuitbreaker-resilience4j</artifactId>
</dependency>
```

2. Enable Circuit Breaker

```
@SpringBootApplication
@EnableCircuitBreaker
public class App { }
```

3. Use @CircuitBreaker

```
@Service
class PaymentService {

    @CircuitBreaker(name = "paymentService", fallbackMethod = "fallback")
    public String callPaymentService() {
        // call external API
        throw new RuntimeException("Service Down");
    }
}
```



```
public String fallback(Exception e) {  
    return "Fallback response";  
}  
}
```

4. Configuration (optional)

```
resilience4j.circuitbreaker.instances.paymentService.failureRateThreshold=50
```

Key Idea

- Detect failures
 - Stop calls temporarily
 - Provide fallback
-

“Circuit Breaker in Spring Boot is implemented using Resilience4j by adding dependency, using @CircuitBreaker annotation, and defining fallback methods to handle failures gracefully.”

Question: How do you configure rate limiting in Spring Boot?

Rate limiting is used to **control how many requests a client can make in a given time.**

1. Using Bucket4j (Common way)

Add dependency

```
<dependency>  
    <groupId>com.bucket4j</groupId>  
    <artifactId>bucket4j-core</artifactId>  
</dependency>
```

Configure rate limit

```
@Component
```

```
class RateLimiterFilter implements Filter {
```

```
    private final Bucket bucket = Bucket4j.builder()  
        .addLimit(Bandwidth.simple(10, Duration.ofMinutes(1))) // 10 requests/min  
        .build();
```

```
    public void doFilter(ServletRequest req, ServletResponse res, FilterChain chain) {  
        if (bucket.tryConsume(1)) {  
            chain.doFilter(req, res); // allow request  
        } else {  
            ((HttpServletResponse) res).setStatus(429); // Too Many Requests  
        }  
    }
```

```
}  
}
```

2. Using API Gateway (Spring Cloud Gateway)

- Configure rate limiting at gateway level (for microservices)

Key Idea

- Limit number of requests
- Prevent abuse and overload

“Rate limiting in Spring Boot can be implemented using libraries like Bucket4j or API Gateway, where requests are restricted based on defined limits such as number of requests per time period.”

Question: How do you secure microservices using API Gateway and JWT?

Overall Idea

- **API Gateway** → single entry point
- **JWT (JSON Web Token)** → used for authentication
- Microservices are protected → only valid requests allowed

Flow

1. User Login

- User sends username & password to **Auth Service**

2. JWT Token Generation

- If valid → server generates **JWT token**

3. Client sends request with token

Authorization: Bearer <token>

4. API Gateway intercepts request

- Extracts JWT token
- Validates:
 - Signature
 - Expiry

5. If token is valid

- Gateway forwards request to microservice

6. If invalid

- Request is rejected (401 Unauthorized)

Example (Gateway Filter)

```
public class JwtFilter implements GatewayFilter {

    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        String token = exchange.getRequest().getHeaders().getFirst("Authorization");

        if (isValid(token)) {
            return chain.filter(exchange);
        } else {
            exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
            return exchange.getResponse().setComplete();
        }
    }
}
```

Key Idea

- JWT → proves identity
 - API Gateway → validates and controls access
 - Microservices → remain secure
-

“Microservices are secured by using an API Gateway that intercepts requests and validates JWT tokens, allowing only authenticated and authorized requests to reach the services.”

Question: What is Spring Boot Actuator?

Spring Boot Actuator is a module used to **monitor and manage a running Spring Boot application**.

What it provides

- Ready-made **endpoints** to check:
 - Application health
 - Metrics
 - Environment details
 - Beans
-

Example endpoints

/actuator/health → app status

/actuator/metrics → performance data

/actuator/env → environment properties

How to enable

<dependency>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-actuator</artifactId>

</dependency>

Key Idea

- Used for **monitoring, debugging, and production support**
-

“Spring Boot Actuator is a module that provides production-ready endpoints to monitor and manage the health, metrics, and configuration of a Spring Boot application.”

Question: What are Actuator endpoints?

Actuator endpoints are predefined URLs provided by Spring Boot Actuator to monitor and manage the application at runtime.

What they provide

- Health status
 - Metrics
 - Application info
 - Environment details
-

Common endpoints

/actuator/health → shows application status

/actuator/metrics → shows performance metrics

/actuator/info → application info

/actuator/env → environment properties

/actuator/beans → list of beans

Key Idea

- Used for **monitoring and debugging in production**
-

“Actuator endpoints are predefined URLs provided by Spring Boot Actuator to monitor and manage application health, metrics, and configuration at runtime.”

Question: What is @SpringBootTest?

@SpringBootTest is used to **load the full Spring Boot application context for testing.**

What it does

- Starts the **entire application context**
 - Allows testing of:
 - Controllers
 - Services
 - Repositories
 - Used for **integration testing**
-

Example

```
@SpringBootTest
class AppTest {

    @Autowired
    private UserService service;

    @Test
    void testService() {
        System.out.println(service.getUsers());
    }
}
```

Key Idea

- Tests application **as a whole (end-to-end)**
-

“@SpringBootTest is used to load the full Spring Boot application context for integration testing of all layers of the application.”

Question: What is Spring Boot DevTools?

Spring Boot DevTools is a module that **improves development experience** by providing features like **automatic restart and live reload**.

Features

1. **Automatic Restart**
 - Application restarts automatically when code changes
 2. **Live Reload**
 - Browser refreshes automatically
 3. **Faster Development**
 - Reduces manual restart time
 4. **Development-only**
 - Not used in production
-

Add dependency

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-devtools</artifactId>
</dependency>
```

Key Idea

- Speeds up development by reducing manual work
-

“Spring Boot DevTools is a development tool that provides automatic restart, live reload, and other features to improve developer productivity during development.

Question: What is Flyway?

Flyway is a tool used for **database migration and version control**.

What it does

- Manages **database schema changes**
 - Applies changes in a **controlled and versioned way**
 - Keeps track of applied scripts
-

How it works

- You create SQL files like:

V1__create_table.sql

V2__add_column.sql

- Flyway runs them in order
 - Stores history in a table (flyway_schema_history)
-

Key Idea

- Version control for database
 - Ensures all environments have same DB structure
-

“Flyway is a database migration tool that manages and applies versioned SQL scripts to keep database schemas consistent across environments.”

Question: What is Liquibase?

Liquibase is a tool used for **database version control and migration**, similar to Flyway.

What it does

- Tracks and manages **database schema changes**
 - Applies changes in a **controlled and versioned way**
 - Ensures consistency across environments
-

How it works

- Uses **changelog files** (XML, YAML, JSON, or SQL)

```
<changeSet id="1" author="dev">
```

```
  <createTable tableName="users">
```

```
    <column name="id" type="int"/>
```

```
    <column name="name" type="varchar(255)"/>
```

```
  </createTable>
```

```
</changeSet>
```

- Liquibase executes changes and keeps history in database

Key Idea

- Version control for database
- Supports multiple formats (not only SQL)

“Liquibase is a database migration tool that manages schema changes using versioned changelog files and ensures consistency across different environments.”

Question: How do Flyway and Liquibase work?

Flyway – Working

1. You create SQL migration files

V1__create_table.sql

V2__add_column.sql

2. On application startup

- Flyway scans migration folder

3. Checks history table

- Table: flyway_schema_history

4. Executes pending scripts

- Runs scripts in **version order (V1 → V2 → V3)**

5. Stores execution record

- Marks scripts as applied
-

Liquibase – Working

1. You create changelog file

```
<changeSet id="1" author="dev">
```

```
  <createTable tableName="users"/>
```

```
</changeSet>
```

2. On application startup

- Liquibase reads changelog

3. Checks database history

- Table: DATABASECHANGELOG

4. Executes new changesets

- Runs only unapplied changes

5. Stores execution record

- Tracks applied changes
-

Key Difference in Working

- Flyway → **SQL-based migrations (versioned files)**
 - Liquibase → **ChangeSets (XML/YAML/JSON/SQL)**
-

“Flyway works by executing versioned SQL scripts in order and tracking them in a history table, whereas Liquibase works by executing changelog files with changesets and tracking them in a database table to ensure consistent schema updates.”

Question: What is @Async?

@Async is used to **run a method asynchronously (in a separate thread)**.

What it does

- Method runs **in background**
 - Main thread does not wait for result
 - Improves performance for long tasks
-

Example

@Service

class MyService {

 @Async

 public void sendEmail() {

 System.out.println("Sending email...");

 }

}

- Method runs in **separate thread**
-

Enable Async

@EnableAsync

@SpringBootApplication

class App {}

Key Idea

- Non-blocking execution
 - Used for tasks like:
 - Email sending
 - File processing
-

“@Async is used to execute methods asynchronously in a separate thread, allowing non-blocking execution and improving application performance.”

Question: What are JWT and Opaque tokens?

JWT (JSON Web Token)

- **Self-contained token**
- Contains user data (claims) inside token
- Structure:
 - **Header**
 - **Payload**
 - **Signature**

👉 Example:

header.payload.signature

- Can be **validated without DB call**
- Faster performance

✓ Pros:

- Stateless
- No need to check DB every time

✗ Cons:

- Cannot easily revoke (until expiry)
-

Opaque Token

- **Random string (no readable data)**
- Does NOT contain user information

👉 Example:

a8sd7f9sdf87sdf

- Server must **check DB or auth server** to validate

✓ Pros:

- Easy to revoke
- More secure (no data exposed)

✗ Cons:

- Requires extra call (slower)
-

Key Difference

- JWT → **contains data, self-validated**
 - Opaque → **no data, requires server validation**
-

“JWT is a self-contained token that stores user information and can be validated without server calls, whereas an opaque token is a random string that requires server-side validation for authentication.”